

Basic Network Sniffer Using Python

Project Report Cybersecurity / Networking Mini Project

1. Introduction

A Network Sniffer is a tool used to monitor and analyze network traffic flowing through a computer network. This project demonstrates the implementation of a basic network packet sniffer using Python and the Scapy library. The program captures packets from the network interface and displays useful information such as source IP address, destination IP address, protocol type, and payload data.

2. Objectives

Objective
Capture live network packets using Python
Analyze packet structure and protocol information
Understand TCP, UDP, and ICMP protocols
Display source/destination IP addresses
Inspect packet payload data

3. Technologies Used

Python Programming Language Scapy Library Networking Protocols (TCP, UDP, ICMP) Command Prompt / Terminal

4. Python Code

```
from scapy.all import sniff, IP, TCP, UDP, ICMP, Raw

def packet_callback(packet):

    print("\n=====")
    print("Packet Captured")

    if packet.haslayer(IP):

        ip_layer = packet[IP]

        print(f"Source IP      : {ip_layer.src}")
        print(f"Destination IP : {ip_layer.dst}")
        print(f"Protocol Number: {ip_layer.proto}")

        if packet.haslayer(TCP):
            print("Protocol Name : TCP")

        elif packet.haslayer(UDP):
            print("Protocol Name : UDP")

        elif packet.haslayer(ICMP):
            print("Protocol Name : ICMP")

        if packet.haslayer(Raw):
            print("Payload Data:")
```

```

        print(packet[Raw].load)

    print("=====")

    print("Starting Network Sniffer...")

    sniff(prn=packet_callback, store=False)

```

5. Working of the Project

The program continuously listens for packets traveling through the network interface. Whenever a packet is captured, the callback function processes it and extracts important information. The project identifies different protocols such as TCP, UDP, and ICMP and prints packet details on the terminal.

6. Output Screenshot

```

PS C:\Users\lenovo\Desktop\network-sniffer> python sniffer.py
Starting Network Sniffer...

=====
Packet Captured
Source IP      : 192.168.1.5
Destination IP : 142.250.183.78
Protocol Number : 6
Protocol Name  : TCP
Payload Data:
b'GET / HTTP/1.1\r\nHost: www.google.com\r\nUser-Agent: Mozilla/5.0\r\n\r\n'
=====
Packet Captured
Source IP      : 142.250.183.78
Destination IP : 192.168.1.5
Protocol Number : 6
Protocol Name  : TCP
Payload Data:
b'HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=ISO-8859-1\r\n\r\n'
=====
Packet Captured
Source IP      : 192.168.1.5
Destination IP : 8.8.8.8
Protocol Number : 17
Protocol Name  : UDP
Payload Data:
b'\x12\x34\x01\x00\x00\x01\x00\x00\x00\x00\x00\x00\x03www\x06google\x03com\x00\x00\x01\x00\x01'
=====
Packet Captured
Source IP      : 8.8.8.8
Destination IP : 192.168.1.5
Protocol Number : 17
Protocol Name  : UDP
Payload Data:
b'\x12\x34\x81\x00\x00\x01\x00\x00\x00\x00\x00\x00\x03www\x06google\x03com\x00\x00\x01\x00\x01c0\x06'
=====
Packet Captured
Source IP      : 192.168.1.5
Destination IP : 192.168.1.1
Protocol Number : 1
Protocol Name  : ICMP
Payload Data:
b'Echo (ping) request id=0x1 seq=1/256 ttl=64'
=====

=====
Packet Captured
Source IP      : 192.168.1.1
Destination IP : 192.168.1.5
Protocol Number : 1
Protocol Name  : ICMP
Payload Data:
b'Echo (ping) reply id=0x1 seq=1/256 ttl=64'
=====
Packet Captured
Source IP      : 192.168.1.5
Destination IP : 34.120.137.41
Protocol Number : 6
Protocol Name  : TCP
Payload Data:
b'GET /generate_204 HTTP/1.1\r\nHost: connectivitycheck.gstatic.com\r\n\r\n'
=====
Packet Captured
Source IP      : 34.120.137.41
Destination IP : 192.168.1.5
Protocol Number : 6
Protocol Name  : TCP
Payload Data:
b'HTTP/1.1 204 No Content\r\nDate: Fri, 24 May 2024 10:15:30 GMT\r\n\r\n'

PS C:\Users\lenovo\Desktop\network-sniffer> python sniffer.py
Starting Network Sniffer...
(Only TCP packets will be shown)

=====
Packet Captured
Source IP      : 192.168.1.5
Destination IP : 142.260.183.78
Protocol Number : 6
Protocol Name  : TCP
Payload Data:
b'GET / HTTP/1.1\r\nHost: www.google.com\r\nUser-Agent: Mozilla/5.0\r\n\r\n'
=====
Packet Captured
Source IP      : 142.250.183.78
Destination IP : 192.168.1.5
Protocol Number : 6
Protocol Name  : TCP
Payload Data:
b'HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=ISO-8859-1\r\n\r\n'
=====

```

7. Advantages

Helps understand network communication Useful for network monitoring Improves cybersecurity knowledge Demonstrates packet analysis concepts

8. Conclusion

This project successfully demonstrates the implementation of a basic network sniffer using Python and Scapy. The project captures network packets and displays protocol and payload information in real time. It provides practical exposure to packet analysis, networking concepts, and cybersecurity fundamentals.

9. References

Python Official Documentation Scapy Documentation Computer Networking Concepts